

DAY 13

Loading CSV data using data ingestion techniques in PySpark. After successfully reading the dataset, I performed various ETL and analytical operations including schema validation, handling null values, removing duplicate records, transforming data using categorical columns, and analyzing revenue-based insights.

DAY 14

loading JSON data through the data ingestion pipeline. The dataset was processed using ETL workflows such as validation, cleaning, date conversion, feature extraction, and analytical tasks including billing analysis, department-wise aggregation, and identifying key business insights from the data.

Why is validation important before analysis?

1. Data Quality Issues Lead to Wrong Results

- Null values can skew aggregations (sum, avg, count)
- Duplicates inflate metrics and create false patterns
- Wrong data types cause calculation errors or runtime failures

2. Silent Failures Are Dangerous

- Spark doesn't always warn about data quality issues
- Analysis on bad data produces incorrect insights
- Business decisions based on flawed analysis can be costly

3. Validation Catches Problems Early

- Schema validation ensures expected columns and types exist
- Null checks identify missing critical data
- Duplicate detection prevents counting the same event multiple times
- Range/format validation catches outliers and malformed data

Validation ensures: Accurate analysis → Correct insights → Better decisions

DAY 15

split()

The `split()` function is used to divide a string into multiple parts based on a delimiter. It is useful when separating words or values stored together in one column.

Example: Splitting a full name into first name and last name.

```
df = df.withColumn("first_name", split(df["full_name"], " ")[0])
```

concat_ws()

The `concat_ws()` function combines multiple columns into a single string using a separator. It is commonly used for joining names or addresses.

Example: Combining first name and last name with a space.

```
df = df.withColumn("full_name", concat_ws(" ", df["first_name"], df["last_name"]))
```

substring()

The `substring()` function extracts a specific part of a string using position and length values. It helps in extracting prefixes, codes, or years from text.

Example: Extracting the first 2 characters from a product code.

```
df = df.withColumn("prefix", substring(df["product_code"], 1, 2))
```

regexp_replace()

The `regexp_replace()` function replaces characters or patterns in a string using regular expressions. It is mainly used for cleaning data.

Example: Removing hyphens from phone numbers.

```
df = df.withColumn("clean_phone", regexp_replace(df["phone"], "-", ""))
```